

StructRL: Structured Action-Space Exploration for Flow-Based VLAs

Anonymous Author(s)

Affiliation

Address

email

Abstract: Flow-based Vision-Language-Action (VLA) models are now widely used for continuous robotic manipulation, and online reinforcement learning (RL) is emerging as a key technique for adapting them to new tasks. Existing RL methods typically inject stochasticity inside the denoising chain, often through isotropic or temporally independent noise. However, effective robot exploration calls for structured noise: temporally smooth and scaled differently across action groups. We show that simply switching the in-chain noise to a structured form does not suffice: noise added at an intermediate flow time can be weakened by the remaining denoising steps before execution, a phenomenon we call *Structured Noise Dilution*. We propose **StructRL**, which avoids dilution by relocating policy stochasticity to the action space via three coupled choices: (i) a deterministic ODE decoder, (ii) structured noise injected directly in the action space, and (iii) last-step replay, where policy-gradient updates avoid assigning likelihoods to intermediate denoising states. This keeps structured exploration tied to the executed action while providing a tractable training signal for the flow decoder. Across three flow-based VLA models on multiple manipulation benchmarks, we empirically show that StructRL improves exploration efficiency and OOD performance over prior in-chain baselines, demonstrating the effectiveness of structured action-space exploration for adapting flow-based VLAs with RL.

Keywords: Flow-based VLA, Reinforcement Learning, Structured Exploration

1 Introduction

Vision-Language-Action (VLA) models cast robot control as instruction-conditioned action prediction, conditioning on visual observations and language instructions while typically pairing pretrained VLM backbones with continuous action heads trained on heterogeneous robot data [1, 2, 3, 4]. Among continuous action-head designs, flow matching [5] has become a common choice in recent VLA policies: it learns a velocity field and supports efficient few-step inference over action chunks. Recent flow-based VLAs such as π_0 [2], $\pi_{0.5}$ [3], and GROOT [4] are trained by large-scale behavior cloning. This imitation objective, however, is fundamentally bounded by the quality and coverage of the offline demonstrations. Improving these pretrained policies beyond the offline data therefore requires feedback from environment interaction, motivating recent RL approaches for flow-based VLAs [6, 7].

In these RL approaches, the exploration distribution determines which actions are evaluated during interaction and therefore shapes what the policy can learn. For continuous robot control, exploratory actions should reflect the structure of robot actions (Fig. 1a). First, actions sampled within a chunk should be temporally smooth, since jittery actions can make execution unstable or unsafe. Second, noise should be scaled differently across action groups, since *position*, *rotation*, and *gripper* actions vary in range and safety sensitivity. Existing RL methods for flow-based VLAs [6, 8, 7] instead inject random noise inside the denoising chain, without explicitly controlling temporal correlation or action-group scale at the action level.

We find that this in-chain design makes structured exploration difficult to realize (Fig. 1b). When structured noise is injected at an intermediate flow time, the remaining denoising steps map the perturbed sample back toward the SFT action manifold. As a result, the temporal and action-group

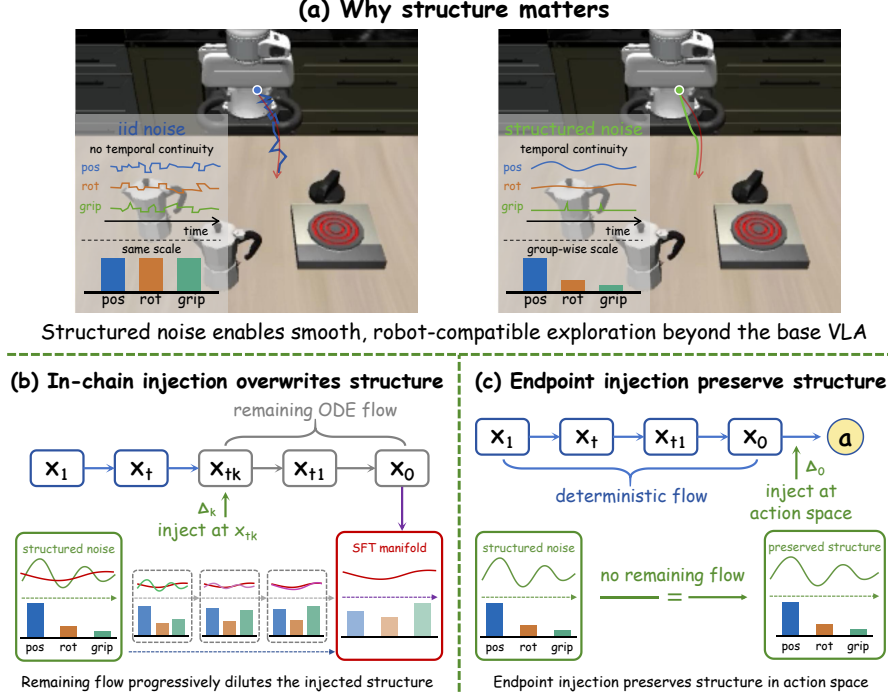


Figure 1: **Structured exploration for flow-based VLA-RL.** (a) Robot exploration benefits from smooth action chunks and group-aware noise scales. (b) In-chain noise can lose its intended structure during the remaining denoising steps. (c) StructRL keeps denoising deterministic and applies structured noise directly in the action space.

structure specified by the injected noise can be weakened or overwritten before the action is executed. We refer to this phenomenon as *Structured Noise Dilution* (Sec. 3).

This observation suggests a simple alternative: keep the flow decoder deterministic and place the exploration distribution directly in the action space (Fig. 1c). We propose **StructRL** based on this idea. Given the clean action chunk produced by ODE denoising, StructRL samples structured action-space noise from an AR(1)-correlated distribution with separate scales for action groups, conditioned on the decoder’s terminal representation. During training, the policy likelihood is evaluated on the sampled action-space noise, so structured exploration stays tied to the executed action rather than to an intermediate denoising state. StructRL then uses last-step replay to pass policy-gradient updates to the flow decoder without defining likelihoods along the denoising chain. Across three flow-based VLA backbones (π_0 , $\pi_{0.5}$, GR00T N1.5) on multiple manipulation benchmarks, StructRL improves exploration efficiency and consistently outperforms prior in-chain baselines on out-of-distribution and long-horizon manipulation tasks. These results suggest that preserving structure in action-space exploration is important for efficient and robust VLA adaptation.

The main contributions can be summarized as follows:

- We identify *Structured Noise Dilution*: structured noise injected inside the flow chain can be weakened by the remaining denoising steps before it reaches the executed action.
- We introduce a action-space RL framework for flow-based VLA with last-step replay mechanism, showing that the flow decoder can be updated through the final Euler step alone, without assigning policy likelihoods to intermediate denoising states.
- Based on this framework, we propose **StructRL**, which injects structured exploration noise directly in the action space using AR(1) temporal correlation and separate scales for position, rotation, and gripper actions.
- Across multiple manipulation benchmarks, we show that StructRL improves exploration efficiency and OOD performance over in-chain baselines.

2 Related Work

Vision-Language-Action Models. Early VLAs such as RT-1 [9], RT-2 [10], OpenVLA [1], and π_0 -FAST [11] represent robot actions as language tokens and train the model autoregressively. Another line keeps the vision-language backbone fixed and trains an attached continuous action head [12, 13]. Recent VLA policies often use flow-matching [5, 14] or diffusion [15, 16] decoders to model continuous action chunks with few-step inference. Our work focuses on RL adaptation of representative flow-based VLAs, including π_0 [2], $\pi_{0.5}$ [3], and GR00T N1.5 [4], which are trained by large-scale imitation learning on heterogeneous robot data.

RL for Generative Robot Policies. RL has been used to optimize generative policies. DDPO [17] treats the denoising process of a diffusion model as a multi-step stochastic policy and applies policy-gradient updates to the denoising chain. In offline continuous-control RL, Diffusion-QL [18], IDQL [19], CPQL [20], and QAM [21] combine generative policy classes with value-based objectives. For flow-based VLAs, π_{RL} [6] introduces two methods: Flow-Noise models denoising as a discrete-time MDP with a learnable noise network, and Flow-SDE converts ODE denoising into an SDE for exploration. Flow-CPS [8] proposes a coefficient-preserving stochastic sampler for RL with flow matching, while π -StepNFT [7] introduces a critic and likelihood-free step-wise negative-aware fine-tuning objective for flow-based VLAs. These works make RL feasible for denoising policies, but their exploration signal is not defined as a structured distribution over executed actions.

Structured Exploration in Continuous-Control RL. Continuous-control RL has explored several ways to structure exploration, including temporally correlated noise across environment steps (OU noise in DDPG [22], colored noise [23, 24]), learned action-dimension coupling [25], state-dependent linear noise [26], parameter-space perturbation [27], and on-manifold exploration [28]. These methods mainly define structure on policy actions or parameters. Flow-based VLAs complicate this design because action samples are produced through a denoising chain, while existing RL methods usually place stochasticity inside the chain. StructRL moves the structured perturbation to the action space and uses last-step replay to pass policy-gradient updates back to the flow decoder.

3 Structured Noise Dilution

This section studies why structure injected inside the denoising chain may not survive to the executed action. We first explain how the remaining denoising steps can filter an intermediate perturbation, then test this effect on a GR00T N1.5 SFT model by injecting noise with controlled temporal correlation and per-group scale at different locations. We finally discuss why the structure induced by the SFT model is not a substitute for task-specific exploration.

Mechanism. In-chain exploration specifies noise before the decoder has finished predicting an action. The remaining denoising steps are still those of the SFT model, so the perturbed state is processed toward demonstration-like action regions. The action-level perturbation is therefore not the injected noise itself: its temporal correlation and action-group scale can be changed by the rest of the decoder, which we call *Structured Noise Dilution*.

Empirical Verification. We test whether Structured Noise Dilution appears in flow-based VLAs, and Table 1 provides direct empirical evidence. Specifically, we use a GR00T N1.5 SFT model on LIBERO-Long [29], and inject structured perturbations Δ with either a prescribed AR(1) coefficient ρ_{inj} along the chunk axis or prescribed per-group magnitudes across position, rotation, and gripper actions. The perturbation is inserted at one of two sites: *in-chain*, before the remaining denoising steps, or *action-space*, after denoising completes. We then measure the structure

Table 1: **Structured noise dilution.** Output statistics after injecting structured noise into a GR00T N1.5 SFT model on LIBERO-Long. We compare action-space injection with in-chain injection under two requested structures: strong temporal correlation ($\rho_{\text{inj}} = 0.95$) and a position-heavy scale ratio $R_{\text{approach}} = (1.0, 0.5, 0.05)$, both using base noise level $\sigma = 0.2$. Action-space injection better follows the requested structure, while in-chain injection stays close to the SFT baseline.

	Action-space			In-chain		
	pos	rot	grip	pos	rot	grip
$\hat{\rho}_{\text{SFT}}$	0.78	0.72	0.57	0.78	0.72	0.57
$\rho_{\text{inj}} = 0.95$	0.70	0.64	0.63	0.79	0.72	0.58
$\hat{\sigma}_{\text{SFT}}$	0.10	0.08	0.21	0.10	0.08	0.21
R_{approach}	0.23	0.13	0.19	0.11	0.08	0.19

of the resulting executed action. The table reports one representative temporal-correlation setting and one representative action-group scale setting. For strong temporal correlation ($\rho_{\text{inj}} = 0.95$), action-space injection changes the measured output correlation, while in-chain injection stays close to the SFT baseline. For the position-heavy scale ratio $R_{\text{approach}} = (1.0, 0.5, 0.05)$ over (position, rotation, gripper), action-space injection increases the position standard deviation from 0.10 to 0.23, whereas in-chain injection changes it only from 0.10 to 0.11. Additional experimental details and full results are provided in App. B.

SFT Structure Constrains Exploration. The structure that survives in-chain injection comes from the SFT model. The SFT-induced correlation $\rho_{\text{SFT}}(o)$ reflects the behavior distribution in the offline data [30, 31], and may be useful as an imitation prior. However, if exploration is mainly shaped by this prior, the policy remains biased toward actions that the SFT model already favors. This makes it harder to reach useful actions outside the SFT prior and can reduce exploration efficiency during RL. These limitations motivate us to move structured exploration to the action space, where temporal correlation and action-group scale can be specified directly.

4 Method: StructRL

We propose StructRL, an action-space RL framework for flow-based VLAs, illustrated in Fig. 2. StructRL keeps the flow decoder deterministic, injects structured noise at the action output, and uses last-step replay to update the decoder from the log-probability of the sampled action-space noise. In this section, we first review flow-based VLA policies and the RL likelihood problem, then describe the action-space RL framework, and finally instantiate the noise distribution with temporal correlation and action-group-aware scaling.

4.1 Preliminaries

Flow-Matching VLAs. A flow-based VLA predicts an action chunk by integrating a learned velocity field $v_\theta(x_t, t, o)$, where o is the multi-modal observation and $x_t \in \mathbb{R}^{C \times D}$ is a noised action chunk. Starting from $x_1 \sim \mathcal{N}(0, I)$, the reverse flow is integrated with T Euler steps:

$$x_{t_k - \delta} = x_{t_k} - \delta v_\theta(x_{t_k}, t_k, o), \quad k = T, T-1, \dots, 1, \quad (1)$$

The final state x_0 is used as the clean action a . We denote by $F_{k \rightarrow 0}^\theta$ the deterministic map from x_{t_k} to x_0 obtained by the remaining k denoising steps.

Policy Likelihood for Flow-based RL. Likelihood-based RL objectives require a tractable policy density, such as policy ratios in PPO [32] or entropy terms in SAC [33]. For a flow-matching VLA, the executed action is obtained after the multi-step denoising process in Eq. 1, so its likelihood is not directly exposed. Existing flow-based VLA RL methods attach likelihoods to denoising-chain transitions [6]. Such chain-level likelihoods are tied to intermediate denoising states, while action-space structured noise is defined on the perturbation added to the executed output action.

4.2 Action-Space RL with Last-Step Replay

Action-space exploration avoids dilution by the remaining denoising steps. Therefore, we first design a framework that can inject noise at the action output while still providing a tractable likelihood for RL. The pretrained flow decoder is run as a deterministic ODE to predict a clean action chunk,

$$x_0^{\text{clean}} = F_{T \rightarrow 0}^\theta(x_1), \quad (2)$$

and then add noise at the action output:

$$a = x_0^{\text{clean}} + \Delta_0. \quad (3)$$

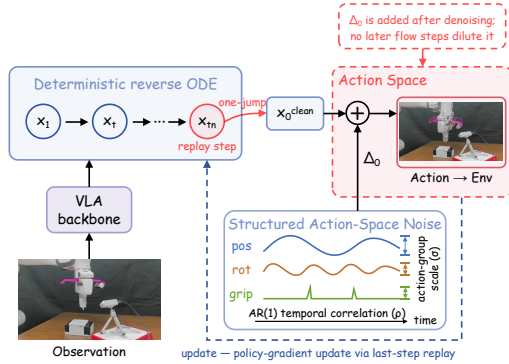


Figure 2: **The StructRL framework.** The reverse flow runs as a deterministic ODE to predict a clean actions x_0^{clean} . StructRL then samples structured noise Δ_0 directly in the action space, with AR(1) temporal correlation and action-group-aware scales. During RL training, the likelihood is evaluated on this action-space noise, and last-step replay passes policy-gradient updates back to the flow decoder.

Since Δ_0 is added after denoising, the likelihood is evaluated on the perturbation applied to the executed action. As a simple example, let $\Delta_0 \sim \mathcal{N}(0, \sigma^2 I)$ in the action space. Its log-probability is

$$\log p_{\text{base}}(\Delta_0) = \sum_{c,d} \log \mathcal{N}\left(\frac{\Delta_{0,c,d}}{\sigma}; 0, 1\right) - CD \log \sigma. \quad (4)$$

To update the flow decoder, we use last-step replay. During rollout, we store the executed action together with a stored late denoising state x_{t_n} . At each policy update, we replay the deterministic jump from t_n to 0:

$$\hat{x}_0^{\text{clean}} = F_{n \rightarrow 0}^\theta(x_{t_n}). \quad (5)$$

In our default setting, x_{t_n} is the last denoising state before the clean action chunk (i.e., $n = 1$). During policy updates, this terminal jump is replayed to compute the log-probability without rerunning the full denoising chain. The action-space noise assigned to the stored action is then

$$\Delta_0^\theta = a - \hat{x}_0^{\text{clean}}, \quad (6)$$

and the policy log-probability is evaluated as

$$\log \pi_\theta(a | o) = \log p_{\text{base}}(\Delta_0^\theta). \quad (7)$$

The likelihood is therefore evaluated for the executed action, rather than for an intermediate denoising-chain transition. This action-level log-probability can be used in standard likelihood-based RL objectives, such as PPO or SAC, with the value or critic loss defined by the chosen algorithm. In simulation, where rollouts can be collected in parallel, we use PPO; in real-world experiments, where rollouts are slow and expensive, we use the more sample-efficient SAC.

Last-step replay updates the decoder through a short terminal denoising jump, while the policy likelihood is evaluated on the action-space noise. This avoids assigning likelihoods to intermediate denoising-chain transitions and provides the basic action-space RL framework used by both the Gaussian baseline and the structured noise in Sec. 4.3. In Sec. 5.3, by setting $T \in \{2, 3, 4, 5\}$, we demonstrate that this framework provides an effective optimization signal for the full flow field.

4.3 Structured Noise Instantiation

We now replace the Gaussian baseline in Sec. 4.2 with a learnable structured distribution $p_\phi(\Delta_0 | h_\theta)$, where $h_\theta \in \mathbb{R}^{C \times d_h}$ is the decoder’s final hidden representation. This distribution makes two properties explicit at the action output: temporal correlation within an action chunk and different noise scales across action groups.

Action-Group-aware Scaling. Robot manipulation policies commonly represent actions as relative end-effector motion, typically covering position, rotation, and gripper control. These action groups have different units, ranges, and safety sensitivity, so a single isotropic noise scale is poorly matched to this action representation. We partition the D action dimensions into three groups, $\mathcal{G} = \{\mathcal{G}_{\text{pos}}, \mathcal{G}_{\text{rot}}, \mathcal{G}_{\text{grip}}\}$. A global base noise level σ^{base} sets the overall noise magnitude, while group-specific bounds control the allowed scale range for each action group. A lightweight MLP head f_ϕ predicts the bounded element-wise multipliers from h_θ :

$$\sigma_{c,d} = \sigma^{\text{base}} \cdot \text{clip}(f_\phi(h_\theta)_{c,d}, [\alpha_{g(d)}, \beta_{g(d)}]), \quad (8)$$

where $g(d)$ is the group of dimension d . The group-specific bounds $[\alpha_g, \beta_g]$ keep the learned scale within a controlled range.

AR(1) Temporal Correlation. Within an action chunk, adjacent actions are executed in order, so independent noise across the chunk may introduce unnecessary jitter. Therefore, we further add temporal structure by sampling an AR(1) noise sequence. For each action dimension d ,

$$\varepsilon_{1,d} \sim \mathcal{N}(0, 1), \quad \varepsilon_{c+1,d} = \rho \varepsilon_{c,d} + \sqrt{1 - \rho^2} \zeta_{c,d}, \quad \zeta_{c,d} \sim \mathcal{N}(0, 1), \quad (9)$$

with $\rho \in [0, 1)$ controlling the temporal correlation. The final action-space perturbation combines the group-aware scale and the AR(1) noise sequence:

$$\Delta_{0,c,d} = \sigma_{c,d} \varepsilon_{c,d}. \quad (10)$$

This construction gives temporally correlated action-space noise with different scales across action groups, making the exploration distribution better aligned with the structure of robot actions.

Table 2: **LIBERO benchmark results.** Average success rates (%) across four task suites, each containing 10 tasks. The few-shot SFT models are directly taken from the π_{RL} checkpoints.

Model	Spatial	Object	Goal	Long	Avg.	Δ Avg.
GR00T N1.5 – Few-shot SFT	41.4	58.6	48.2	61.9	52.5	—
+ π_{RL} (Flow-SDE+PPO)	96.6	100	93.8	95.6	96.5	+44.0
+ Baseline	96.2	99.4	91.8	95.6	95.8	+43.3
+ StructRL	99.2	100	97.6	99.0	99.0	+46.3
π_0 – Few-shot SFT	65.3	64.4	49.8	51.2	57.6	—
+ π_{RL} (Flow-SDE+PPO)	98.4	99.4	96.2	90.2	96.0	+38.4
+ π -StepNFT	93.5	98.0	83.7	86.7	90.5	+32.9
+ Baseline	98.8	99.0	97.2	90.2	96.3	+38.7
+ StructRL	98.8	99.8	98.4	91.8	97.2	+39.6
$\pi_{0.5}$ – Few-shot SFT	84.6	95.4	84.6	43.9	77.1	—
+ π_{RL} (Flow-SDE+PPO)	99.6	100	98.8	93.0	97.9	+20.8
+ π -StepNFT	97.8	100	98.2	79.8	94.0	+16.9
+ Baseline	99.4	100	93.8	86.0	94.8	+17.7
+ StructRL	99.6	100	98.8	90.2	97.2	+20.1

Structured log-probability. Given the replayed action-space perturbation Δ_0^θ , we evaluate its likelihood under the generative rule in Eq. 10. The AR(1) density is applied to the corresponding noise coordinates, with the scale change-of-variables term:

$$\log p_\phi(\Delta_0^\theta | h_\theta) = \sum_{d=1}^D \left[\log \mathcal{N}(\varepsilon_{1,d}^\theta; 0, 1) + \sum_{c=1}^{C-1} \log \mathcal{N}(\varepsilon_{c+1,d}^\theta; \rho \varepsilon_{c,d}^\theta, 1 - \rho^2) \right] - \sum_{c,d} \log \sigma_{c,d}, \quad (11)$$

where $\varepsilon_{c,d}^\theta = \Delta_{0,c,d}^\theta / \sigma_{c,d}$ is used only for likelihood evaluation: it is the AR(1) noise value corresponding to the replayed perturbation under the current scale. This structured likelihood replaces Eq. 4 inside the same action-space RL framework. We provide the detailed derivation in App. C.

Regularization. The learned scale σ controls action-space exploration, so unconstrained updates can collapse or over-amplify the perturbations. Therefore, we regularize σ with three lightweight terms: a dimension-entropy term that discourages scale collapse onto a few action dimensions, a budget term that keeps the overall noise magnitude near the target level, and a temporal smoothness term that avoids abrupt scale changes along the chunk. The full form is given in App. D.

5 Experiments

5.1 Experimental Setup

Benchmarks. We conduct experiments on three standard simulation benchmarks: LIBERO [29], ManiSkill [34], and CALVIN [35]. Hyperparameter settings and CALVIN results are provided in App. G and App. F, respectively. All experiments are conducted on 4 NVIDIA A800 GPUs.

Baseline variants. We adopt π_0 [2], $\pi_{0.5}$ [3], and GR00T N1.5 [4] as the base VLA models throughout the experiments. For simulation, we train the RL variants with PPO, since the environments allow parallel rollout collection. StructRL is compared against several baselines: the SFT model, π_{RL} [6] (Flow-SDE+PPO), and π -StepNFT [7]. Flow-SDE injects noise inside the denoising chain and trains with PPO, while π -StepNFT keeps SDE-based in-chain exploration but replaces likelihood-based policy-gradient training with a step-wise fine-tuning objective. We define the version that adds Gaussian noise within our action-space RL framework as *Baseline*. StructRL further replaces this Gaussian noise with structured action-space noise.

5.2 Main Results

Results on LIBERO. Table 2 presents the results across the four LIBERO suites. On GR00T N1.5, where the few-shot SFT model reaches only 52.5 % average, StructRL pushes the average success rate to 99.0 %, outperforming Flow-SDE+PPO by 2.5% with consistent gains on the four suits. On π_0 , StructRL still yields a modest improvement against Flow-SDE+PPO. The Baseline variant (Gaussian noise) consistently lags behind StructRL, confirming that structured noise provides additional benefits.

Table 3: **ManiSkill benchmark results.** Success rates (%) on in-distribution (IND) tasks and three OOD splits: Vision (5 tasks), Semantic (5 tasks), and Execution (2 tasks). Avg. reports the mean success rate across the OOD splits.

Model	IND	OOD			
		Vision	Semantic	Execution	Avg.
π_0 – SFT	38.4	32.6	8.4	13.2	18.1
+ π_{RL} (Flow-SDE+PPO)	78.8	61.1	25.4	31.5	39.3
+ π -StepNFT	79.2	69.1	49.1	33.1	50.4
+ Baseline	84.1	79.4	63.3	59.1	67.3
+ StructRL	87.8	80.0	64.2	56.2	66.8
$\pi_{0.5}$ – SFT	40.1	38.8	16.6	22.3	25.9
+ π_{RL} (Flow-SDE+PPO)	90.9	68.0	34.5	45.4	49.3
+ π -StepNFT	85.4	76.9	56.6	45.1	59.5
+ Baseline	85.9	76.2	62.2	59.4	65.9
+ StructRL	90.9	78.3	68.4	64.2	70.3

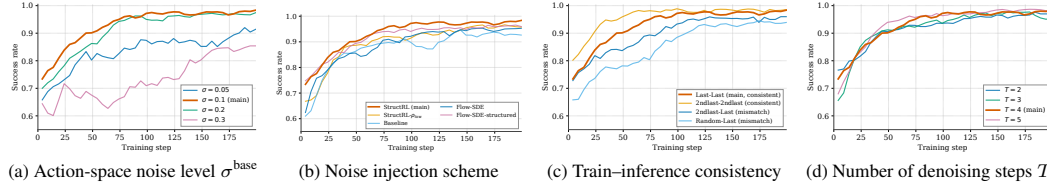


Figure 3: **Four ablations on LIBERO-Long with GR00T N1.5.** (a) Moderate action-space noise gives stable online improvement. (b) Structured action-space noise explores more efficiently than Gaussian or in-chain variants. (c) Matched train–inference replay improves performance, with larger matched jumps further improving exploration efficiency. (d) Last-step replay mechanism provides an effective optimization signal for the full flow field across different denoising steps.

235 **Results on ManiSkill.** Table 3 reports the simulated results on the ManiSkill benchmark. StructRL
236 achieves the highest in-distribution success rate of 87.8% on π_0 , and matches the best on $\pi_{0.5}$.
237 For OOD evaluation, the action-space RL framework shows a clear advantage over Flow-SDE’s
238 in-chain noise injection. Compared with Flow-SDE+PPO, StructRL improves the OOD average
239 from 39.3% to 66.8% on π_0 and from 49.3% to 70.3% on $\pi_{0.5}$, with similar gains observed for
240 the Baseline variant. These results indicate that action-space exploration can preserve strong in-
241 domain performance while enabling exploration beyond the narrow SFT manifold, thereby yielding
242 substantially better generalization.

243 5.3 Ablation Studies

244 All ablations use GR00T N1.5 with PPO on LIBERO-Long, the LIBERO suite that most directly
245 tests long-horizon execution. For a fair comparison, all runs use the same 200 training iterations and
246 are evaluated every 5 iterations (Fig. 3).

247 **(a) Sensitivity to the Noise Level σ^{base} .** We set $\sigma^{\text{base}} \in \{0.05, 0.1, 0.2, 0.3\}$ to test how the magnitude
248 of exploration noise affects training stability. As shown in Fig. 3(a), small noise levels under-explore,
249 while large noise levels over-perturb the executed actions and destabilize training. The intermediate
250 setting provides learnable signals over a broader action region, enabling efficient online improvement.

251 **(b) Noise Injection Scheme.** We compare four ways of injecting exploration noise: *action-space*
252 *Gaussian noise* (i.e., the baseline in this paper), *StructRL*, *StructRL*– ρ_{low} (reduced temporal correla-
253 tion), *Flow-SDE* (isotropic in-chain noise), and *Flow-SDE*–*structured* (same structure noise as
254 StructRL injected in-chain). Fig. 3(b) reveals two findings. First, simply adding noise in the action
255 space is not enough: isotropic Gaussian noise converges more slowly than Flow-SDE. We attribute
256 this to the fact that exploration guided by the SFT manifold structure is often more effective than
257 isotropic random action-space perturbations. Second, structured noise in the action space shows
258 higher exploration efficiency than structured exploration in-chain, and richer structured noise infor-

mation generally enables more efficient exploration. This suggests that StructRL has greater potential for exploring useful smooth action signals beyond the SFT manifold.

(c) Train–Inference Consistency. Last-step replay allows action-space RL to update the flow field by replaying a terminal ODE jump during training. This gives a flexible setting: the replay can cover one or more final denoising steps, and the inference-time ODE step can either match or differ from the training replay. We therefore compare four configurations. **Last-Last:** last-step replay ($n = 1$) in training and last-step inference. **2ndlast-2ndlast:** second-last-step replay ($n = 2$) in training and second-last-step inference. **2ndlast-Last:** second-last-step replay ($n = 2$) in training but last-step inference. **Random-Last:** randomly selected replay depth in training and last-step inference. The first two settings are consistent between training and inference, while the last two deliberately introduce mismatch. As shown in Fig. 3(c), the train–inference consistent settings substantially outperform the mismatched settings. We also find that a larger replay jump brings a more evident gain in exploration efficiency.

(d) Robustness Evaluation. To evaluate the robustness of last-step replay mechanism for our action-space RL framework, we vary the number of denoising steps as $T \in \{2, 3, 4, 5\}$. As shown in Fig. 3(d), all settings show similar convergence speeds and achieve comparable final performance. **This result shows that last-step replay can route gradients only through a terminal denoising jump while still providing an efficient optimization signal for the full flow field.**

5.4 Real-World Experiment

We directly apply StructRL with sample-efficient SAC in real-world *plug charger in* task, using $\pi_{0.5}$ as the base model. This task requires precise alignment and insertion: after SFT with 40 demonstrations, the model achieves only a 10% success rate. StructRL then performs zero-shot online RL directly, using few human interventions during training to collect samples, and reaches an 80% success rate within two hours, demonstrating the effectiveness of our method in real-world adaptation.

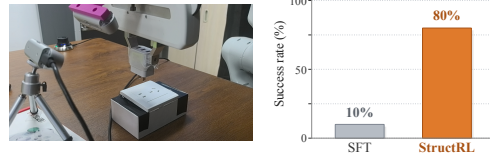


Figure 4: **Real-world experiment.** The left image shows the setup for the *plug charger in* task. The right plot compares the success rate of the SFT model and StructRL after online adaptation with few human interventions.

6 Conclusion

We presented StructRL, an action-space RL framework for RL adaptation of flow-based VLAs. Motivated by structured noise dilution, StructRL keeps the flow decoder deterministic, injects temporally correlated and group-aware noise on the executed action chunk, and uses last-step replay to update the flow field from action-space likelihoods. Experiments across multiple manipulation benchmarks show improved exploration efficiency and OOD performance over in-chain noise baselines, and a real-world charger insertion task further demonstrates that StructRL enables effective online RL adaptation of a pretrained VLA.

7 Limitations

The main claim in this paper relies on the assumption that an SFT flow-based VLA forms a relatively concentrated action distribution, and that later denoising steps can pull intermediate perturbations back toward this distribution. Structured noise dilution should therefore be viewed as an empirical explanation for current mainstream flow-based VLAs, rather than a formal guarantee for all flow decoders or samplers. In methods, StructRL currently uses AR(1) temporal correlation and group-aware Gaussian noise. This distribution is suitable for smooth continuous exploration, but it may still be insufficient for tasks that require multimodal strategy switches or large deviations from the SFT manifold. In experiments, our real-world evaluation is limited to one precision manipulation task and still relies on a small number of human interventions to collect online samples. Future work can evaluate StructRL on more real-world tasks and robot platforms, and explore richer, safer, and more adaptive action-space exploration distributions.

References

- [1] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi, Q. Vuong, et al. Openvla: An open-source vision-language-action model. In *Conference on Robot Learning*, pages 2679–2713. PMLR, 2025.
- [2] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [3] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [4] J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- [5] Y. Lipman, R. T. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.
- [6] K. Chen, Z. Liu, T. Zhang, Z. Guo, S. Xu, H. Lin, H. Zang, X. Li, Q. Zhang, Z. Yu, et al. π_1 : Online rl fine-tuning for flow-based vision-language-action models. *arXiv preprint arXiv:2510.25889*, 2025.
- [7] S. Wang, X. Wang, Z. Zhu, M. Pei, X. Cui, C. Deng, J. Zhao, G. Huang, H. Zhang, and J. Wang. π -stepnft: Wider space needs finer steps in online rl for flow-based vlas. *arXiv preprint arXiv:2603.02083*, 2026.
- [8] F. Wang and Z. Yu. Coefficients-preserving sampling for reinforcement learning with flow matching. *arXiv preprint arXiv:2509.05952*, 2025.
- [9] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. Robotics: Science and Systems Foundation, 2023.
- [10] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.
- [11] K. Pertsch, K. Stachowicz, B. Ichter, D. Driess, S. Nair, Q. Vuong, O. Mees, C. Finn, and S. Levine. Fast: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*, 2025.
- [12] O. M. Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.
- [13] M. J. Kim, C. Finn, and P. Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.
- [14] X. Liu, C. Gong, and Q. Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.
- [15] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025.

- [16] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu. 3d diffusion policy: Generalizable visuomotor policy learning via simple 3d representations. Robotics: Science and Systems Foundation, 2024.
- [17] K. Black, M. Janner, Y. Du, I. Kostrikov, and S. Levine. Training diffusion models with reinforcement learning. In *International Conference on Learning Representations*, volume 2024, pages 4965–4987, 2024.
- [18] Z. Wang, J. J. Hunt, and M. Zhou. Diffusion policies as an expressive policy class for offline reinforcement learning. *arXiv preprint arXiv:2208.06193*, 2022.
- [19] P. Hansen-Estruch, I. Kostrikov, M. Janner, J. G. Kuba, and S. Levine. Idql: Implicit q-learning as an actor-critic method with diffusion policies. *arXiv preprint arXiv:2304.10573*, 2023.
- [20] Y. Chen, H. Li, and D. Zhao. Boosting continuous control with consistency policy. In *Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems*, pages 335–344, 2024.
- [21] Q. Li and S. Levine. Q-learning with adjoint matching. *arXiv preprint arXiv:2601.14234*, 2026.
- [22] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra. Continuous control with deep reinforcement learning. In *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, 2016.
- [23] O. Eberhard, J. Hollenstein, C. Pinneri, and G. Martius. Pink noise is all you need: Colored noise exploration in deep reinforcement learning. In *The Eleventh International Conference on Learning Representations*, 2023.
- [24] J. Hollenstein, G. Martius, and J. Piater. Colored noise in ppo: Improved exploration and performance through correlated action sampling. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 12466–12472, 2024.
- [25] A. S. Chiappa, A. Marin Vargas, A. Huang, and A. Mathis. Latent exploration for reinforcement learning. *Advances in Neural Information Processing Systems*, 36:56508–56530, 2023.
- [26] A. Raffin, J. Kober, and F. Stulp. Smooth exploration for robotic reinforcement learning. In *Conference on robot learning*, pages 1634–1644. PMLR, 2022.
- [27] M. Plappert, R. Houthoofd, P. Dhariwal, S. Sidor, R. Y. Chen, X. Chen, T. Asfour, P. Abbeel, and M. Andrychowicz. Parameter space noise for exploration. In *International Conference on Learning Representations*, 2018.
- [28] Y. Jin, J. Lv, H. Xue, W. Chen, C. Wen, and C. Lu. Soe: Sample-efficient robot policy self-improvement via on-manifold exploration. *arXiv preprint arXiv:2509.19292*, 2025.
- [29] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.
- [30] S. Ross, G. Gordon, and D. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 627–635. JMLR Workshop and Conference Proceedings, 2011.
- [31] C. G. Atkeson and S. Schaal. Robot learning from demonstration. In *International Conference on Machine Learning (ICML)*, 1997.
- [32] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

- [33] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pages 1861–1870. Pmlr, 2018.
- [34] S. Tao, F. Xiang, A. Shukla, Y. Qin, X. Hinrichsen, X. Yuan, C. Bao, X. Lin, Y. Liu, T.-K. Chan, et al. Maniskill3: Gpu parallelized robot simulation and rendering for generalizable embodied ai. In *7th Robot Learning Workshop: Towards Robots with Human-Level Abilities*, 2025.
- [35] O. Mees, L. Hermann, E. Rosete-Beas, and W. Burgard. Calvin: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. *IEEE Robotics and Automation Letters*, 7(3):7327–7334, 2022.

A StructRL Training Pseudocode

This appendix provides the training pseudocode for the framework introduced in Sec. 4.2. Algorithm 1 summarizes the action-space RL framework used by both the Gaussian baseline and StructRL. During rollout, the flow decoder first predicts a clean action through the deterministic ODE, and exploration noise is then added in the action space. During policy updates, last-step replay recomputes the clean action from a stored late denoising state, so the likelihood can be evaluated on the action-space perturbation assigned to the executed action.

Algorithm 1 StructRL

Require: Flow decoder F^θ initialized from the SFT model; action-space noise distribution p ; likelihood-based RL objective $\mathcal{L}_{\text{RL}}$

- 1: **for** each rollout iteration **do**
- 2: Sample $x_1 \sim \mathcal{N}(0, I)$ and run the flow decoder as a deterministic ODE.
- 3: Store a late denoising state x_{t_n} and obtain $x_0^{\text{clean}} = F_{T \rightarrow 0}^\theta(x_1)$.
- 4: Sample action-space noise $\Delta_0 \sim p$, where p is either the Gaussian baseline or the structured distribution defined by Eq. 10.
- 5: Execute $a = x_0^{\text{clean}} + \Delta_0$ and store the transition, including a , x_{t_n} , and the rollout log-probability when required by the RL objective.
- 6: **end for**
- 7: **for** each policy update **do**
- 8: Replay the terminal denoising jump from the stored state: $\hat{x}_0^{\text{clean}} = F_{n \rightarrow 0}^\theta(x_{t_n})$.
- 9: Recover the replayed action-space perturbation $\Delta_0^\theta = a - \hat{x}_0^{\text{clean}}$.
- 10: Evaluate the action-level log-probability using Eq. 4 for the Gaussian baseline or Eq. 11 for StructRL.
- 11: Optimize $\mathcal{L}_{\text{RL}}$ with this action-level log-probability.
- 12: **end for**

B Full Empirical Verification of Structured Noise Dilution

This appendix expands the empirical verification in Sec. 3. Table 1 in the main paper reports one representative temporal-correlation setting and one representative action-group scale setting. Here we provide the full sweep at $\sigma = 0.2$ (Table A1) and an additional run at $\sigma = 0.5$ (Table A2). The larger noise level checks that the observed dilution is not an artifact of using a small perturbation.

Setup. We use a GR00T N1.5 SFT model on LIBERO-Long and inject structured perturbations Δ in two ways. For temporal structure, Δ is sampled with a prescribed AR(1) coefficient ρ_{inj} along the chunk axis. For action-group scale, Δ uses target magnitudes $\sigma_{\text{inj}} = \sigma \cdot R$ over the (position, rotation, gripper) groups, where σ is the base noise level and R is a ratio vector. We evaluate three ratios: $R_{\text{approach}} = (1.0, 0.5, 0.05)$ (position-heavy), $R_{\text{pregrasp}} = (0.05, 0.1, 1.0)$ (gripper-heavy), and $R_{\text{carry}} = (0.5, 0.5, 0.1)$ (position+rotation, low gripper). The perturbation is inserted either *in-chain*, before the remaining denoising steps, or in the *action space*, after denoising completes. We then measure the lag-1 autocorrelation $\hat{\rho}_{\text{out}}$ and per-group standard deviation $\hat{\sigma}_{\text{out}}$ of the executed action.¹

The SFT baseline rows report the unperturbed reverse flow.

¹All measurements average over 256 Monte-Carlo perturbation samples and 16 test-set observations.

Table A1: **Full per-condition data at base noise level $\sigma = 0.2$.** GR00T N1.5 SFT model on LIBERO-Long. **(a)** Lag-1 autocorrelation $\hat{\rho}_{out}$ under different injected ρ_{inj} . **(b)** Per-group standard deviation $\hat{\sigma}_{out}$ under different magnitude profiles (target $\sigma_{inj} = 0.2 \cdot R$). Action-space injection tracks the requested structure, while in-chain injection stays close to the SFT baseline.

Injected structure	Action-space output			In-chain output		
	pos	rot	grip	pos	rot	grip
<i>(a) Lag-1 autocorrelation $\hat{\rho}_{out}$</i>						
SFT baseline (ρ_{SFT} , no injection)	0.784	0.718	0.566	0.784	0.718	0.566
$\rho_{inj} = 0.00$ (no banding)	0.115	-0.033	0.225	0.785	0.718	0.564
$\rho_{inj} = 0.30$	0.310	0.203	0.378	0.787	0.721	0.562
$\rho_{inj} = 0.70$	0.556	0.493	0.560	0.788	0.725	0.577
$\rho_{inj} = 0.95$ (strong banding)	0.705	0.643	0.634	0.789	0.723	0.577
output range (max-min over $\rho_{inj} \in [0, 0.95]$)	0.590	0.676	0.409	0.004	0.007	0.015
<i>(b) Per-group standard deviation $\hat{\sigma}_{out}$ (target $\sigma_{inj} = 0.2 \cdot R$)</i>						
SFT baseline (σ_{SFT} , no injection)	0.105	0.077	0.206	0.105	0.077	0.206
$R_{approach} = (1.0, 0.5, 0.05)$ (pos-heavy)	0.227	0.128	0.192	0.109	0.078	0.193
$R_{pregrasp} = (0.05, 0.1, 1.0)$ (grip-heavy)	0.106	0.080	0.314	0.106	0.077	0.214
$R_{carry} = (0.5, 0.5, 0.1)$ (pos+rot)	0.147	0.127	0.205	0.106	0.077	0.190
output range (max-min across the three ratios)	0.121	0.048	0.122	0.003	0.001	0.024

Table A2: **Full per-condition data at base noise level $\sigma = 0.5$.** Same GR00T N1.5 SFT model on LIBERO-Long as Table A1, with a larger base noise level (target $\sigma_{inj} = 0.5 \cdot R$). The same dilution pattern appears: action-space injection follows the requested structure, while in-chain injection remains close to the SFT baseline.

Injected structure	Action-space output			In-chain output		
	pos	rot	grip	pos	rot	grip
<i>(a) Lag-1 autocorrelation $\hat{\rho}_{out}$</i>						
SFT baseline (ρ_{SFT} , no injection)	0.780	0.723	0.584	0.780	0.723	0.584
$\rho_{inj} = 0.00$ (no banding)	-0.038	-0.061	0.275	0.785	0.737	0.589
$\rho_{inj} = 0.30$	0.192	0.181	0.390	0.797	0.752	0.597
$\rho_{inj} = 0.70$	0.489	0.484	0.558	0.812	0.772	0.612
$\rho_{inj} = 0.95$ (strong banding)	0.644	0.635	0.609	0.803	0.759	0.617
output range (max-min over $\rho_{inj} \in [0, 0.95]$)	0.682	0.696	0.334	0.027	0.035	0.028
<i>(b) Per-group standard deviation $\hat{\sigma}_{out}$ (target $\sigma_{inj} = 0.5 \cdot R$)</i>						
SFT baseline (σ_{SFT} , no injection)	0.105	0.077	0.208	0.105	0.077	0.208
$R_{approach} = (1.0, 0.5, 0.05)$ (pos-heavy)	0.511	0.262	0.209	0.116	0.080	0.199
$R_{pregrasp} = (0.05, 0.1, 1.0)$ (grip-heavy)	0.109	0.093	0.572	0.107	0.077	0.249
$R_{carry} = (0.5, 0.5, 0.1)$ (pos+rot)	0.273	0.262	0.206	0.109	0.081	0.197
output range (max-min across the three ratios)	0.402	0.169	0.366	0.009	0.004	0.052

Note on Monte-Carlo fluctuation. The SFT baseline rows should be the same across the $\sigma = 0.2$ and $\sigma = 0.5$ runs because they involve no injected perturbation. The small numerical differences in the tables (e.g., $\hat{\rho}_{SFT, pos} = 0.7840$ at $\sigma = 0.2$ vs. 0.7799 at $\sigma = 0.5$) come from independent Monte-Carlo estimates. With 256 perturbation samples and 16 observations, the standard error is larger than these small shifts, so they do not affect the conclusions.

Temporal correlation is diluted in-chain. Table A1(a) reports $\hat{\rho}_{out}$ as ρ_{inj} sweeps from 0 to 0.95. Under in-chain injection, the output correlation stays close to the SFT baseline $\rho_{SFT} \approx (0.78, 0.72, 0.57)$ for all injected values, changing by less than 0.02 in each group. Under action-space injection, the output correlation changes with the requested value; for example, position correlation increases from 0.11 to 0.70 as ρ_{inj} increases from 0 to 0.95. This shows that temporal structure can be specified directly in the action space, while the remaining denoising steps largely absorb the same structure when it is injected in-chain.

Per-group magnitude follows the same pattern. Table A1(b) repeats the comparison for group-wise magnitude profiles. Action-space injection follows the requested group scale: the position-heavy profile increases $\hat{\sigma}_{pos}$ from 0.105 to 0.227, while the gripper-heavy profile increases $\hat{\sigma}_{grip}$ from 0.206

to 0.314. In-chain injection changes the same statistics only slightly. The in-chain output is still anisotropic, but the anisotropy mostly reflects the SFT model rather than the injected profile.

Robustness across base noise levels. Table A2 repeats the protocol at $\sigma = 0.5$, which is $2.5\times$ larger than the setting in Table A1. The qualitative pattern remains the same. Action-space injection still tracks both the requested temporal correlation and the requested group scale. In-chain injection changes the executed action statistics only mildly; for example, the position-heavy profile increases $\hat{\sigma}_{\text{pos}}$ from 0.105 to 0.116, far below the action-space value of 0.511. This confirms that structured noise dilution is not caused only by a small base noise level.

C Log-Probability of Structured Action-Space Noise

This appendix derives Eq. 11. The sampling process first draws an AR(1) noise sequence for each action dimension d :

$$\varepsilon_{1,d} \sim \mathcal{N}(0, 1), \quad \varepsilon_{c+1,d} = \rho \varepsilon_{c,d} + \sqrt{1 - \rho^2} \zeta_{c,d}, \quad \zeta_{c,d} \sim \mathcal{N}(0, 1), \quad c = 1, \dots, C-1. \quad (12)$$

The action-space perturbation is obtained by applying the learned diagonal scale:

$$\Delta_{0,c,d} = \sigma_{c,d} \varepsilon_{c,d}. \quad (13)$$

The density of the AR(1) sequence factorizes as

$$\log p(\varepsilon) = \sum_{d=1}^D \left[\log \mathcal{N}(\varepsilon_{1,d}; 0, 1) + \sum_{c=1}^{C-1} \log \mathcal{N}(\varepsilon_{c+1,d}; \rho \varepsilon_{c,d}, 1 - \rho^2) \right]. \quad (14)$$

During last-step replay, the stored action and the replayed clean action define the perturbation Δ_0^θ . Under the current scale, its corresponding AR(1) coordinate is

$$\varepsilon_{c,d}^\theta = \frac{\Delta_{0,c,d}^\theta}{\sigma_{c,d}}. \quad (15)$$

The transformation from ε to Δ_0 is diagonal, with Jacobian determinant $\prod_{c,d} \sigma_{c,d}$. The change of variables gives

$$\log p_\phi(\Delta_0^\theta | h_\theta) = \log p(\varepsilon^\theta) - \sum_{c,d} \log \sigma_{c,d}, \quad (16)$$

which yields Eq. 11. This derivation only describes likelihood evaluation for the replayed perturbation. Sampling still follows the forward construction $\varepsilon \rightarrow \Delta_0$ in Eq. 10.

D Regularizer for the Structured Noise Module

This appendix details the regularizer for the structured noise module in Sec. 4.3. The learned scale σ controls the magnitude of action-space exploration. To keep it in a useful range during RL, we use three lightweight penalties:

$$\mathcal{L}_{\text{reg}}(\sigma) = \lambda_{\text{ent}}(\log D - \mathcal{H}(\bar{\sigma})) + \lambda_{\text{b}}(\|\sigma\|_F - \sigma^{\text{base}} \sqrt{CD})^2 + \lambda_{\text{s}} \sum_{c=1}^{C-1} \|\sigma_{c+1} - \sigma_c\|^2, \quad (17)$$

where $\bar{\sigma}_{c,d} = \sigma_{c,d} / \sum_{d'} \sigma_{c,d'}$ and $\mathcal{H}(\bar{\sigma}) = -\sum_d \bar{\sigma}_{c,d} \log \bar{\sigma}_{c,d}$, averaged over batch and chunk positions. The dimension-entropy term discourages scale collapse onto a few dimensions, the magnitude-anchor term keeps the total exploration scale near the target level, and the smoothness term avoids abrupt scale changes along the chunk.

E Real-World Experimental Details

This appendix provides additional details for the real-world experiment in Sec. 5.4.

Scene setup. The experiment is conducted on a Franka Emika Panda robot with a ROS 2 control stack. The scene contains a power strip and a plug fixed to the robot gripper. The goal is to insert the plug into the power strip, which requires precise end-effector positioning and contact control. The policy observes the scene through a third-person camera.

Algorithm. We instantiate the action-space RL framework with SAC for this experiment, using $\pi_{0.5}$ as the base model. SAC is chosen because real-world rollouts are slow and sample efficiency is more important than large-scale parallel data collection.

Training setup. We use a dense reward defined as the negative distance to the target end-effector pose, together with a task success bonus of 3. To reduce safety risks from excessive downward exploration, we monitor and constrain the force along the z axis. We find that SFT on a small number of real-world samples is not sufficient to improve performance and can instead increase unsafe downward exploration. Therefore, we perform zero-shot online RL from the $\pi_{0.5}$ base model and use a small number of human interventions at the beginning of training to quickly add positive signals to the replay buffer.

F Long-Horizon Chaining on CALVIN

This appendix presents the CALVIN long-horizon evaluation referenced in Sec. 5. CALVIN evaluates long-horizon behavior by chaining up to five language instructions in one episode. This setting stresses the temporal axis of StructRL because jitter or unstable exploration in an early subtask can affect later subtasks. Table F1 reports per-length success on $\pi_{0.5}$. StructRL achieves the highest average sequence length and is the only method above 0.93 at Len4. The gap over the time-independent in-chain baselines becomes larger as the sequence length increases, which is consistent with the role of action-space temporal correlation.

Table F1: **Long-horizon chaining on CALVIN.** Per-length success rate (fraction in $[0, 1]$) and average sequence length on $\pi_{0.5}$. StructRL achieves the highest average and maintains the strongest performance at longer sequence lengths.

Model	Len1	Len2	Len3	Len4	Len5	Avg.
$\pi_{0.5}$ – SFT	0.927	0.843	0.767	0.688	0.613	3.838
+ π_{RL} (Flow-SDE+PPO)	0.997	0.982	0.958	0.910	0.870	4.717
+ π_{RL} (Flow-Noise+PPO)	0.996	0.976	0.939	0.896	0.845	4.652
+ Baseline	0.998	0.990	0.964	0.929	0.868	4.749
+ StructRL	1.000	0.993	0.968	0.934	0.880	4.775

G Training Hyperparameters

Table G1: **Training hyperparameters for LIBERO** across the three VLA backbones and four task suites.

Parameters	LIBERO											
	GR00T N1.5				π_0				$\pi_{0.5}$			
	Spatial	Object	Goal	Long	Spatial	Object	Goal	Long	Spatial	Object	Goal	Long
Train epochs	200	200	200	200	200	200	400	400	400	400	400	400
Batch size	1024	1024	1024	1024	2048	2048	2048	2048	2048	2048	2048	2048
Update epochs	1	1	1	1	1	1	4	4	1	1	4	4
Actor lr	2e-5	2e-5	2e-5	2e-5	2e-5	2e-5	5e-6	5e-6	5e-6	5e-6	5e-6	5e-6
Value lr	2e-5	2e-5	2e-5	2e-5	2e-5	2e-5	1e-4	1e-4	1e-4	1e-4	1e-4	1e-4
Noise lr	2e-5	2e-5	2e-5	2e-5	2e-5	2e-5	1e-4	1e-4	1e-4	1e-4	1e-4	1e-4
Interaction steps	240	240	320	480	240	240	320	480	240	240	320	480
Parallel environments	64	64	64	64	64	64	64	64	64	64	64	64
Rollout epochs	8	8	8	8	8	8	8	8	8	8	8	8
Action chunk C	5	5	5	10	5	5	5	10	5	5	5	10
Denoise steps	4	4	4	4	4	4	4	4	3	5	5	5
Noise level σ	0.2	0.2	0.2	0.2	0.2	0.1	0.1	0.1	0.1	0.05	0.05	0.05
Structured noise	AR(1) ρ (pos, rot, grip): (0.8, 0.6, 0.7) Clip scale (pos/rot/grip): (0.1, 2.0)/(0.1, 1.5)/(0.1, 3.0)											

Table G2: Training hyperparameters for CALVIN and ManiSkill multitask settings.

Parameters	CALVIN	ManiSkill	
	$\pi_{0.5}$	π_0	$\pi_{0.5}$
Train epochs	300	400	400
Batch size	2048	5120	5120
Update epochs	4	4	5
Actor lr	5e-6	2e-5	2e-5
Value lr	1e-4	2e-5	2e-5
Noise lr	1e-4	2e-5	2e-5
Interaction steps	480	80	80
Parallel environments	64	320	320
Rollout epochs	8	1	1
Action chunk H	5	5	5
Denoise steps	5	4	4
Noise level σ	0.2	0.2	0.2
Structured noise	AR(1) ρ : (0.8, 0.6, 0.7) Clip scale s : (0.1, 2.0)/(0.1, 1.5)/(0.1, 3.0)		